

# The Case for Learned Index Structures

Tim Kraska

Alex Beutel

Ed H. Chi

Jeffrey Dean

Neoklis Polyzotis

SIGMOD'18

# Background

- Efficient data access → index structures
- **HOWEVER**
  - Do not take advantage of more common patterns prevalent in real-world data
  - → Can we learn these patterns to create more efficient index structures? **Learned Indices**

# Goal

- Learn a model that encapsulates data patterns to enable the creation of specialized index structures → **learned indices**
- Scope
  - Match the semantic guarantees of pre-existing indexing structures
    - Range indices
    - Point indices
    - Existence indices
  - Demonstrate reduced computational cost compared to baselines
  - Achieve improvement on CPU
- **Key observation:** Data structures can be decomposed into **learned model** and **auxiliary structures**

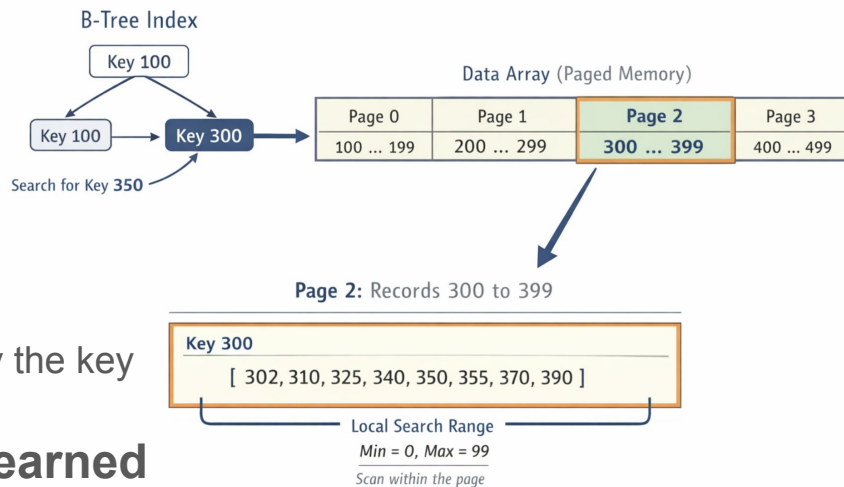
# Range Indices

- “Predict” location of a value within a key-sorted set

- Ex. B-Trees

- Key-value pairs stored in tree-like structure
- Each node has capacity N
- All keys in a node are stored **sorted**
- Key lookup = which range is my key in
- Common practice:
  - Divide records into pages and index only the key of every nth record

- We can match/outperform B-Tree with **Learned Indices**



# B-Tree vs Learned Index

- B-Tree  $\approx$  regression tree  $\therefore$  model
- Can functionally cover the guarantees of B-Trees
  - B-Trees can be replaced with any regression model
    - Linear regression
    - Neural networks

| B-Tree                                                   | ML Model                           |
|----------------------------------------------------------|------------------------------------|
| strong min- and max-error guarantee over the stored keys | Same error guarantees once trained |
| Need to be rebalanced                                    | Need to be retrained               |
| <b>log(n)</b> lookup                                     | <b>Constant</b> time lookup        |
| Hard to parallelize<br><b>Cannot</b> leverage GPU        | <b>Can</b> leverage GPU            |

# Range Indices: CDF

Observation: a model that predicts the position given a key inside a sorted array effectively approximates the CDF where the CDF is given as:

$$p = F(\text{Key}) * N$$

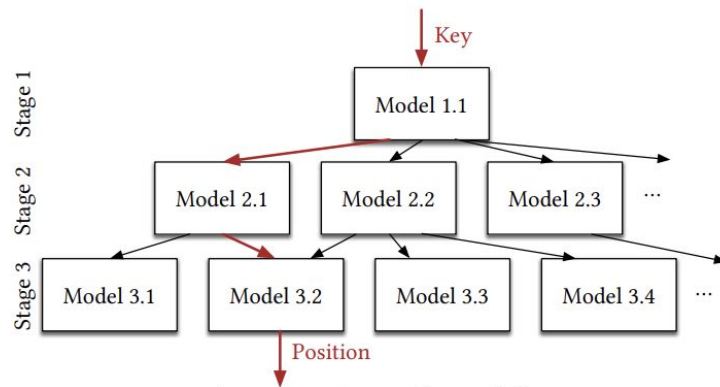
- $p$ : position estimate
- $F(\text{Key})$ : Estimated CDF of observing smaller key than lookup key [ $P(X \leq \text{Key})$ ]
- $N$ : Total # keys

# Implementation: Recursive-Model Indexes(RMI)

- Recursive reduction of prediction space
  - Based on mixture of experts work
- Benefits
  - Separates model size and complexity from execution cost
  - Leverages ease of learning distribution
  - Easier to each achieve “last-mile accuracy”
  - No in-between search process

Loss depends on previous model output

$$L_\ell = \sum_{(x, y)} (f_\ell^{(\lfloor M_\ell f_{\ell-1}(x)/N \rfloor)}(x) - y)^2$$



Roughly predicts position

→ position is used to pick next smaller model

$$L_0 = \sum_{(x, y)} (f_0(x) - y)^2$$

# Hybrid Indexes and Searching

- Can combine different types of models to learn different distributions
  - Ex. top level ReLU NN with bottom level simple linear regression models
  - Ex. B-Trees as bottom as fallback
    - **Bounds worst-case performance to B-Tree**
- Learned indexes return positions, not regions
  - Leveraged in search strategies
    - Model Biased Search: Binary search with middle point predicted by model
    - Biased Quaternary Search:
      - Three split points instead of one
      - Focus search around prediction

---

**Algorithm 1: Hybrid End-To-End Training**

---

```
Input: int threshold, int stages[], NN_complexity
Data: record data[], Model index[][]
Result: trained index
1  $M = \text{stages.size};$ 
2  $\text{tmp\_records}[][];$ 
3  $\text{tmp\_records}[1][1] = \text{all\_data};$ 
4 for  $i \leftarrow 1$  to  $M$  do
5     for  $j \leftarrow 1$  to  $\text{stages}[i]$  do
6          $\text{index}[i][j] = \text{new NN trained on tmp\_records}[i][j];$ 
7         if  $i < M$  then
8             for  $r \in \text{tmp\_records}[i][j]$  do
9                  $p = \text{index}[i][j](r.\text{key}) / \text{stages}[i + 1];$ 
10                 $\text{tmp\_records}[i + 1][p].\text{add}(r);$ 
11 for  $j \leftarrow 1$  to  $\text{index}[M].\text{size}$  do
12      $\text{index}[M][j].\text{update\_err}(\text{tmp\_records}[M][j]);$ 
13     if  $\text{index}[M][j].\text{max\_abs\_err} > \text{threshold}$  then
14          $\text{index}[M][j] = \text{new B-Tree trained on tmp\_records}[M][j];$ 
15 return index;
```

---



# Training and Evaluation

- Train quickly(minutes per model)

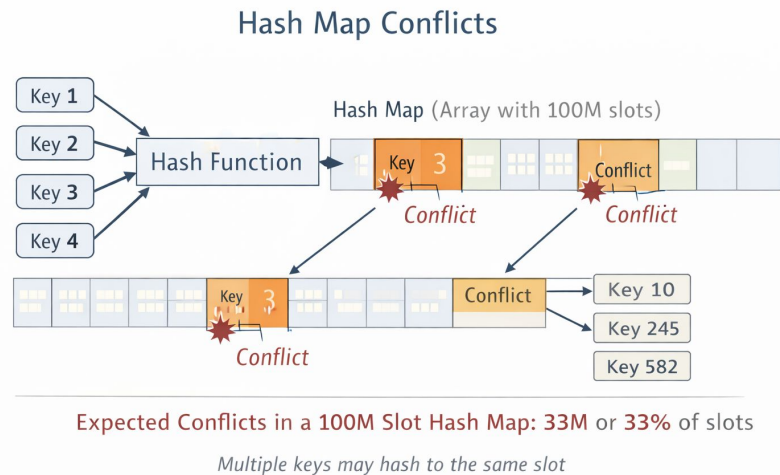
| Type          | Config                 | Map Data      |             |             | Web Data      |             |             | Log-Normal Data |             |             |
|---------------|------------------------|---------------|-------------|-------------|---------------|-------------|-------------|-----------------|-------------|-------------|
|               |                        | Size (MB)     | Lookup (ns) | Model (ns)  | Size (MB)     | Lookup (ns) | Model (ns)  | Size (MB)       | Lookup (ns) | Model (ns)  |
| Btree         | page size: 32          | 52.45 (4.00x) | 274 (0.97x) | 198 (72.3%) | 51.93 (4.00x) | 276 (0.94x) | 201 (72.7%) | 49.83 (4.00x)   | 274 (0.96x) | 198 (72.1%) |
|               | page size: 64          | 26.23 (2.00x) | 277 (0.96x) | 172 (62.0%) | 25.97 (2.00x) | 274 (0.95x) | 171 (62.4%) | 24.92 (2.00x)   | 274 (0.96x) | 169 (61.7%) |
|               | page size: 128         | 13.11 (1.00x) | 265 (1.00x) | 134 (50.8%) | 12.98 (1.00x) | 260 (1.00x) | 132 (50.8%) | 12.46 (1.00x)   | 263 (1.00x) | 131 (50.0%) |
|               | page size: 256         | 6.56 (0.50x)  | 267 (0.99x) | 114 (42.7%) | 6.49 (0.50x)  | 266 (0.98x) | 114 (42.9%) | 6.23 (0.50x)    | 271 (0.97x) | 117 (43.2%) |
|               | page size: 512         | 3.28 (0.25x)  | 286 (0.93x) | 101 (35.3%) | 3.25 (0.25x)  | 291 (0.89x) | 100 (34.3%) | 3.11 (0.25x)    | 293 (0.90x) | 101 (34.5%) |
| Learned Index | 2nd stage models: 10k  | 0.15 (0.01x)  | 98 (2.70x)  | 31 (31.6%)  | 0.15 (0.01x)  | 222 (1.17x) | 29 (13.1%)  | 0.15 (0.01x)    | 178 (1.47x) | 26 (14.6%)  |
|               | 2nd stage models: 50k  | 0.76 (0.06x)  | 85 (3.11x)  | 39 (45.9%)  | 0.76 (0.06x)  | 162 (1.60x) | 36 (22.2%)  | 0.76 (0.06x)    | 162 (1.62x) | 35 (21.6%)  |
|               | 2nd stage models: 100k | 1.53 (0.12x)  | 82 (3.21x)  | 41 (50.2%)  | 1.53 (0.12x)  | 144 (1.81x) | 39 (26.9%)  | 1.53 (0.12x)    | 152 (1.73x) | 36 (23.7%)  |
|               | 2nd stage models: 200k | 3.05 (0.23x)  | 86 (3.08x)  | 50 (58.1%)  | 3.05 (0.24x)  | 126 (2.07x) | 41 (32.5%)  | 3.05 (0.24x)    | 146 (1.79x) | 40 (27.6%)  |

|      | Lookup Table<br>w/ AVX search | FAST    | Fixe-Size Btree<br>w/ interpol. search | Multivariate<br>Learned Index |
|------|-------------------------------|---------|----------------------------------------|-------------------------------|
| Time | 199 ns                        | 189 ns  | 280 ns                                 | 105 ns                        |
| Size | 16.3 MB                       | 1024 MB | 1.5 MB                                 | 1.5 MB                        |

|               | Config                 | Size(MB)      | Lookup (ns)  | Model (ns) |
|---------------|------------------------|---------------|--------------|------------|
| Btree         | page size: 32          | 13.11 (4.00x) | 1247 (1.03x) | 643 (52%)  |
|               | page size: 64          | 6.56 (2.00x)  | 1280 (1.01x) | 500 (39%)  |
|               | page size: 128         | 3.28 (1.00x)  | 1288 (1.00x) | 377 (29%)  |
|               | page size: 256         | 1.64 (0.50x)  | 1398 (0.92x) | 330 (24%)  |
| Learned Index | 1 hidden layer         | 1.22 (0.37x)  | 1605 (0.80x) | 503 (31%)  |
|               | 2 hidden layers        | 2.26 (0.69x)  | 1660 (0.78x) | 598 (36%)  |
| Hybrid Index  | t=128, 1 hidden layer  | 1.67 (0.51x)  | 1397 (0.92x) | 472 (34%)  |
|               | t=128, 2 hidden layers | 2.33 (0.71x)  | 1620 (0.80x) | 591 (36%)  |
|               | t= 64, 1 hidden layer  | 2.50 (0.76x)  | 1220 (1.06x) | 440 (36%)  |
|               | t= 64, 2 hidden layers | 2.79 (0.85x)  | 1447 (0.89x) | 556 (38%)  |
| Learned QS    | 1 hidden layer         | 1.22 (0.37x)  | 1155 (1.12x) | 496 (43%)  |

# Point Indices

- Point lookups
- Deterministically map key to value
- Challenge: prevent collisions/conflicts
  - Can be addressed with:
    - Linked lists, secondary probing, bucketing
- Solution:
  - ML model to reduce conflicts
  - **Hash Model Index**



# Implementation: Hash Model Index

Observation: Learning CDF of key distribution = learning better hash function

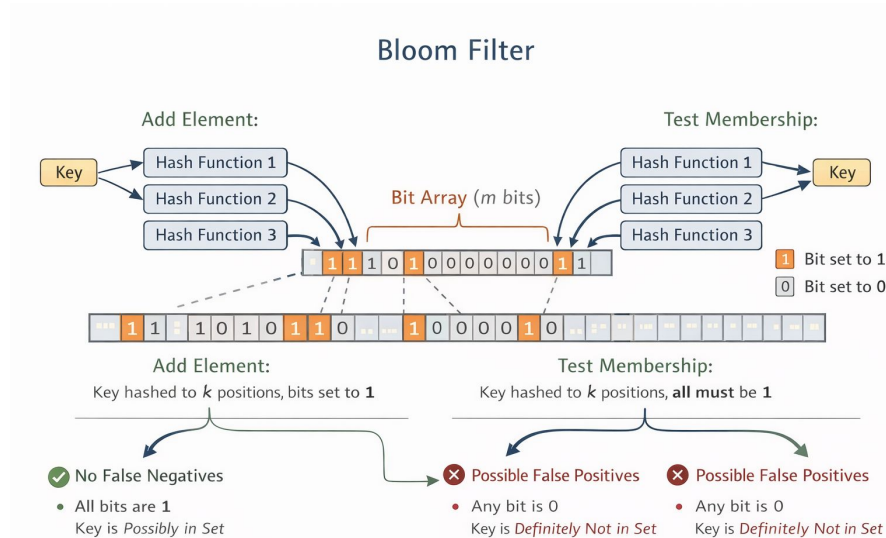
$$h(K) = F(K) * M,$$

- Leverages RMI architecture
- Inserts, look-ups, and conflicts depend on Hash map architecture
- Derived benefits depend on:
  - How well CDF is modeled
  - Hash map architecture

|            | % Conflicts Hash Map | % Conflicts Model | Reduction |
|------------|----------------------|-------------------|-----------|
| Map Data   | 35.3%                | 07.9%             | 77.5%     |
| Web Data   | 35.3%                | 24.7%             | 30.0%     |
| Log Normal | 35.4%                | 25.9%             | 26.7%     |

# Existence Indices

- Specifically Bloom Filters
  - Space efficient** probabilistic data structure to test whether an element is a member of a set
  - Commonly used to check existence in cold storage → higher latency
- FNR = 0
- A good hash function for a Bloom filter
  - Lots of collisions among keys
  - Lots of collisions among non-keys
  - No collisions of keys and non-keys

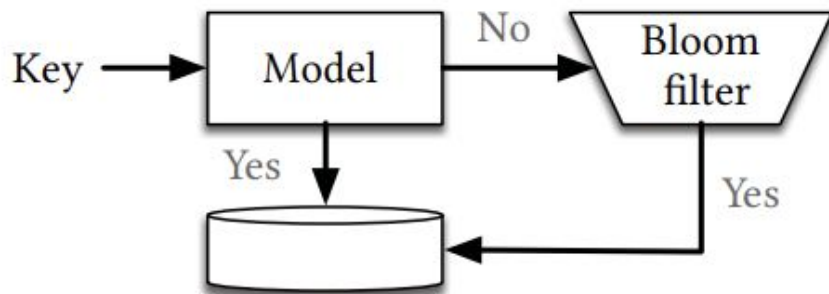


# Learned Bloom Filters as a Classification Problem

- Existence index = binary probabilistic classification task
- Sigmoid activation for RNN with loss:

$$L = \sum_{(x,y) \in \mathcal{D}} y \log f(x) + (1 - y) \log(1 - f(x)).$$

- Likely FNR  $\neq 0$ 
  - Overflow Bloom filter (see right)
    - Cases where  $f(x) < \tau$
    - Scales with FNR

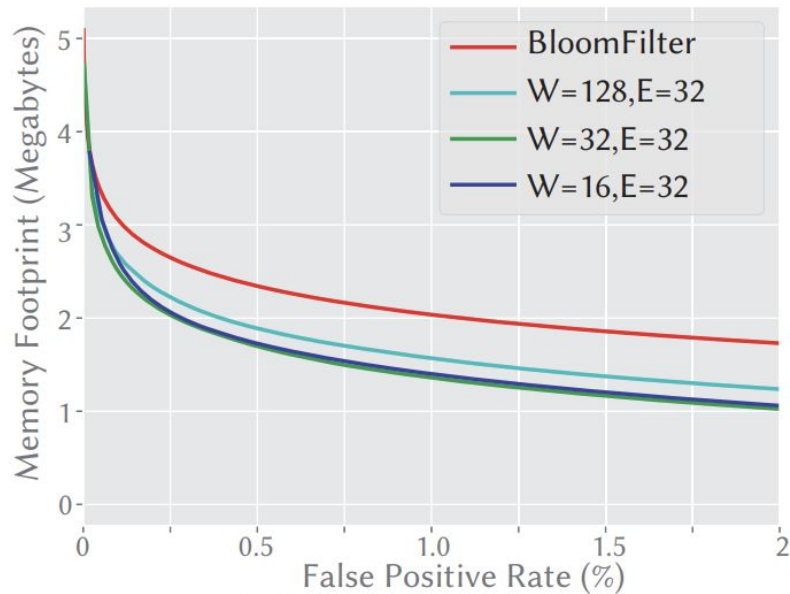


# Learned Bloom Filters with Model Hashes

- Learn hash function to **increase collisions among keys** and **increase collisions among non-keys**
  - Use same probabilistic classification model
- Hash function:  $d = \lfloor f(x) * m \rfloor$ 
  - d: hash function
  - M: size of array
  - Keys mapped to higher range of bit positions, non-keys mapped to lower range

# Learned Bloom Filters Results

- Higher model accuracy = better savings on Bloom Filter size
- Model does not need to use same features as Bloom Filter





# Next Steps

- More tightly integrate the existing hardware-conscious index strategies with learned models
- Index compression(ex. Dictionary compression)
- More effectively modeling the CDF
- More model types, multi-dimensional indexes, speed up sorting